Zephyr Project Overview

A proven RTOS ecosystem, by developers, for developers

Agenda

1 Zephyr overview

2 Alternative Flow: Command Line Only

Compare and Contrast – OS vs RTOS vs Bare-Metal

- **Bare-metal offers the most control over the operation of the application**
  - This comes at the expense of frameworks and ease-of-use
  - Not tuned for performance, memory, or power

- **An RTOS is tuned to run tasks quickly and sequentially**
  - This optimization tends to lead to higher per-task performance
  - Also leads to lowered power and memory requirements

- **An operating system is tuned to run multiple tasks in parallel**
  - Per-task performance typically slower than either bare-metal or RTOS
  - Support for large amounts of memory (via virtualization)
  - Support for complex peripherals & device drivers

FreeRTOS Overview

- FreeRTOS originally adopted by AMD/Xilinx in 2014
  - It was non-proprietary with an active community
- FreeRTOS is often used as a pathway for FuSa via SafeRTOS
  - SafeRTOS is a safety certified version of the FreeRTOS codebase
  - SafeRTOS is provided by Wittenstein (High Integrity Systems)
  - https://www.highintegritysystems.com/safertos/
- FreeRTOS was acquired by Amazon in 2017
  - Now available as a public branch of AWS FreeRTOS
  - There is now not as much community support

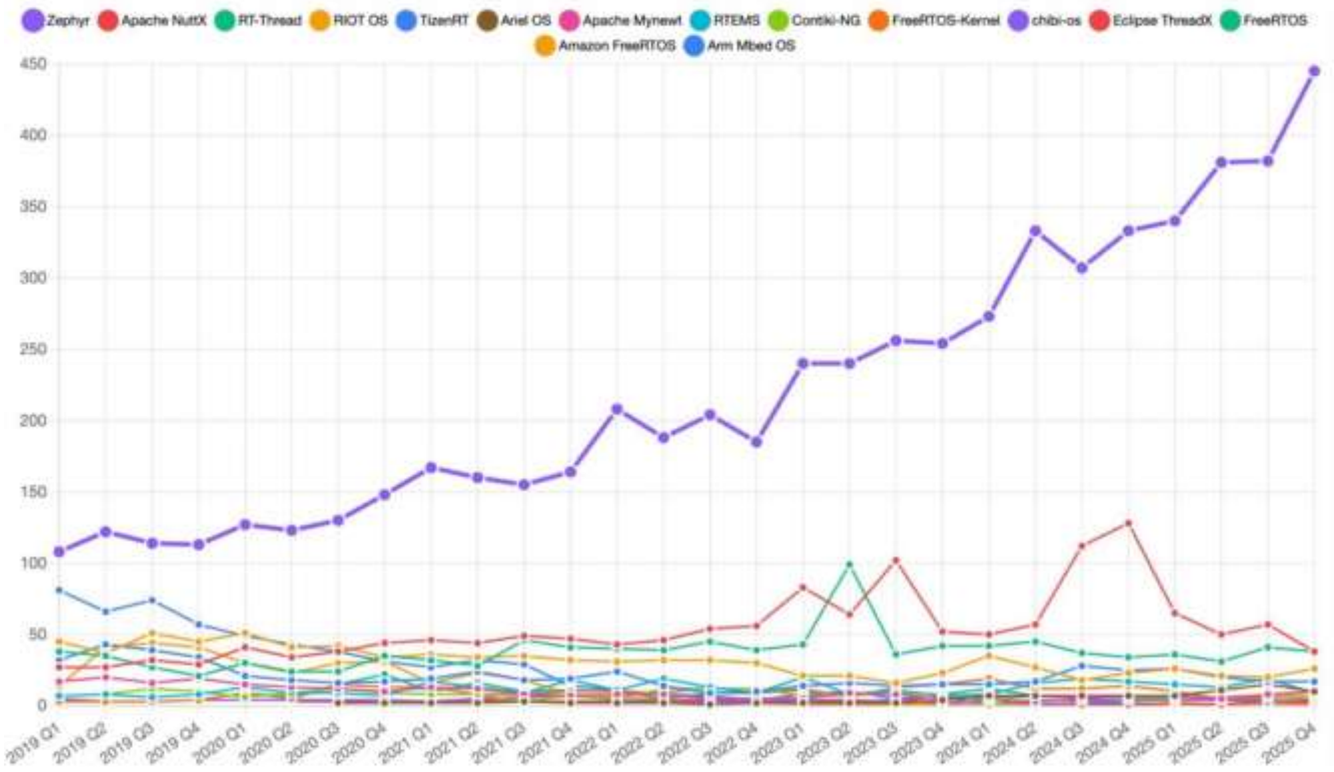
Zephyr® Overview

- Linux foundation open source project
  - Modern open source RTOS implementation
  - Strong community support and documentation
  - Has gained a lot of momentum
- Highly configurable and modular
  - Devicetree based architecture makes supporting new boards quick and easy
  - Rich networking support: IPv6, IPv4, UDP, TCP, ICMPv4, and ICMPv6 out of the box
- Robust and secure kernel
  - Support for multiple scheduling algorithms
  - Native memory protection and fault handling

A “Linux-Like” RTOS

- Developing using Zephyr® is much more intuitive for embedded developers coming from the Linux world compared to other RTOS options
- Complete development environment
  - This is rare in the RTOS world
- Portable (HAL)
- POSIX compliance
- Configuration uses Kconfig
  - Menu- and GUI-based configuration options
- Filesystem support

Unique Contributers per Month



Use Cases for a Real Time OS

What is Zephyr?

Zephyr

AVNET

SMALL *yet* SCALABLE

< 8KB Flash

< 5KB RAM

from small sensor nodes

... to complex multi-core systems

FLEXIBLE *yet* SECURE

Heavily customizable

Out-of-the-box support for 1000 boards and 100s of sensors

Built with safety & security in mind

Certification-ready

Long-term Support

OPEN-SOURCE

Permissively licensed (Apache 2.0)

Vendor-neutral governance

ECOSYSTEM

Vibrant community

Supported by major silicon vendors

Products running Zephyr Today


Oticon More Hearing Aid


Lildog & Lilcat Pet Tracker


Livestock Tracker


Moto Watch 100


Samsung Galaxy Ring


Proglove


Adhoc Smart Waste


Google Chromebook


Framework laptop


Keeb.io BDN9


Hati-ACE


Blackhole™ PCIe AI Accelerator


BLiXT solid state circuit breaker


Aethero Deimos Satellite

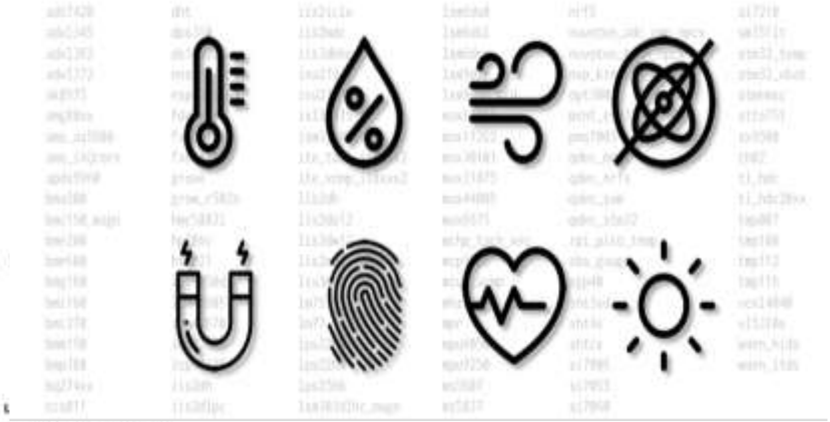

PHYTEC Distancer


Laird Connectivity sensors & gateways


BeST pump monitoring


Vestas Wind Turbines

Hardware: 1000+ Boards & Sensoren



Supported Hardware Architectures

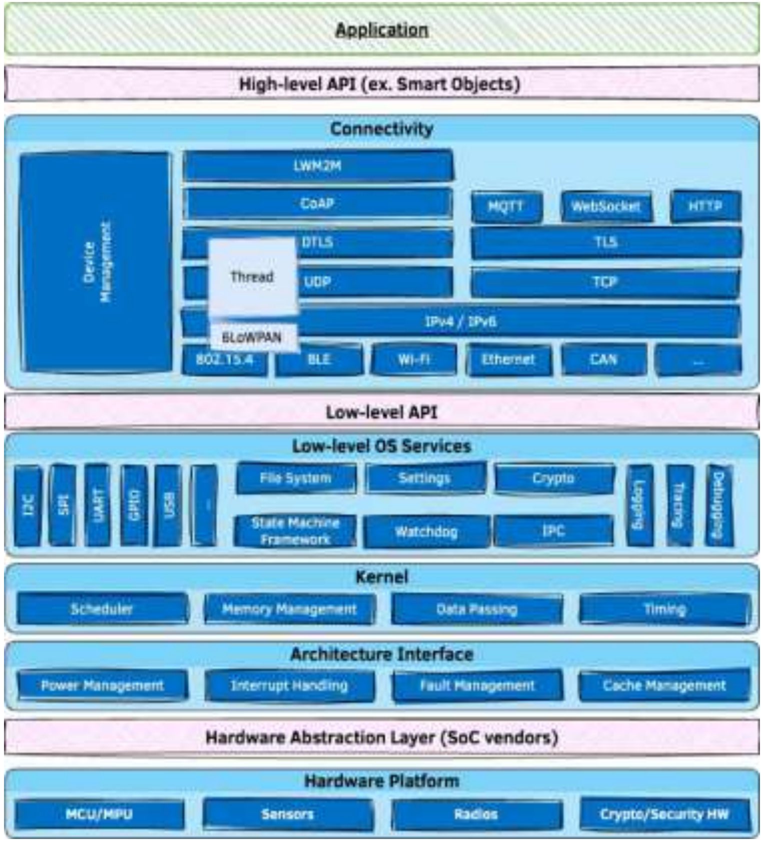
Cortex-M, Cortex-R
& Cortex-A

x86 & x86_64

32 & 64 bit

Xtensa

Architecture



Zephyr & Security

Weekly Coverity scans

MISRA scans

Automated Code checks per pull request

[Documented secure Coding practices]

Vulnerability response criteria publicly documented

SBOM generation per [ISO/IEC 5962:2021]

Native Network Stack

- Built from scratch, on top of Zephyr
- native kernel concepts

- Dual mode **IPv4/IPv6 stack**
- DHCP v4, IPv4 autoconf, IPv6 SLAAC,
- DNS, SNTP

- Multiple network interfaces support
- Time Sensitive Networking support
- **BSD Sockets**-based API
- Supports IP offloading
- Compliance and security tested

Bluetooth 5.3 compliant •
LE Controller • Host • Mesh •
Bluetooth-SIG qualifiable

USB 2.0 • USB-C •
Device & Host • WebUSB

Devicetree

- Describe & configure the available hardware on the target system
- Decouple the application from the hardware
- Same mechanism as on Embedded Linux Systems

```
&i2c1 {
    pinctrl-0 = <&i2c1_scl_pb8 &i2c1_sda_pb9>;
    pinctrl-names = "default";
    clock-frequency = <I2C_BITRATE_FAST>;
    status = "okay";

    lsm6dsl@6a {
        compatible = "st,lsm6dsl";
        reg = <0x06a >;
    };

    hts221@5f {
        compatible = "st,hts221";
        reg = <0x5f >;
    };

    // …
};
```

.dts file example

Go West

- **West is a meta tool which can:**
- Manage your Zephyr project and their underlying Git repos
- Build Zephyr apps
- Flash boards
- Help you debug Zephyr apps
- Be extended with your own custom commands (Python-based)

6/24/2026

Sources & Links

DOKU & CODE

1 Offizielle Dokumentation
docs.zephyrproject.org

2 Getting Started Guide
docs.zephyrproject.org/latest/develop/getting_started

3 Alle Boards (1072 und mehr)
docs.zephyrproject.org/latest/boards/

4 Quellcode auf GitHub
github.com/zephyrproject-rtos/zephyr

SECURITY & COMMUNITY

5 Security & CVE-Advisories
github.com/zephyrproject-rtos/zephyr/security

6 SBOM erzeugen (west spdx)
https://docs.zephyrproject.org/latest/develop/west/zephyr-cmds.html#software-bill-of-materials-west-spdx

7 Projekt & Community (Discord)
https://discord.com/invite/Ck7jw53nU2